Unit 5
FIFOs and Streaming Interfaces

EL-GY 9463: INTRODUCTION TO HARDWARE DESIGN

PROFS. SUNDEEP RANGAN, SIDDHARTH GARG

Learning Objectives

- ❑ Describe the operation and work out the timing of FIFO reads and writes
- ❑ Describe and simulate the AXI4-Streaming protocol
  - Pure streaming interfaces
  - Packetized streaming interfaces
- ❑ Design simple streaming modules in Vitis HLS
- ❑ Design robust command and response packets with:
  - Transactions IDs for detecting missed packets and error codes
- ❑ Write serialization and deserialization methods
  - Manually and using an interface description language
- ❑ Implement packet-based streaming modules in Vitis HLS

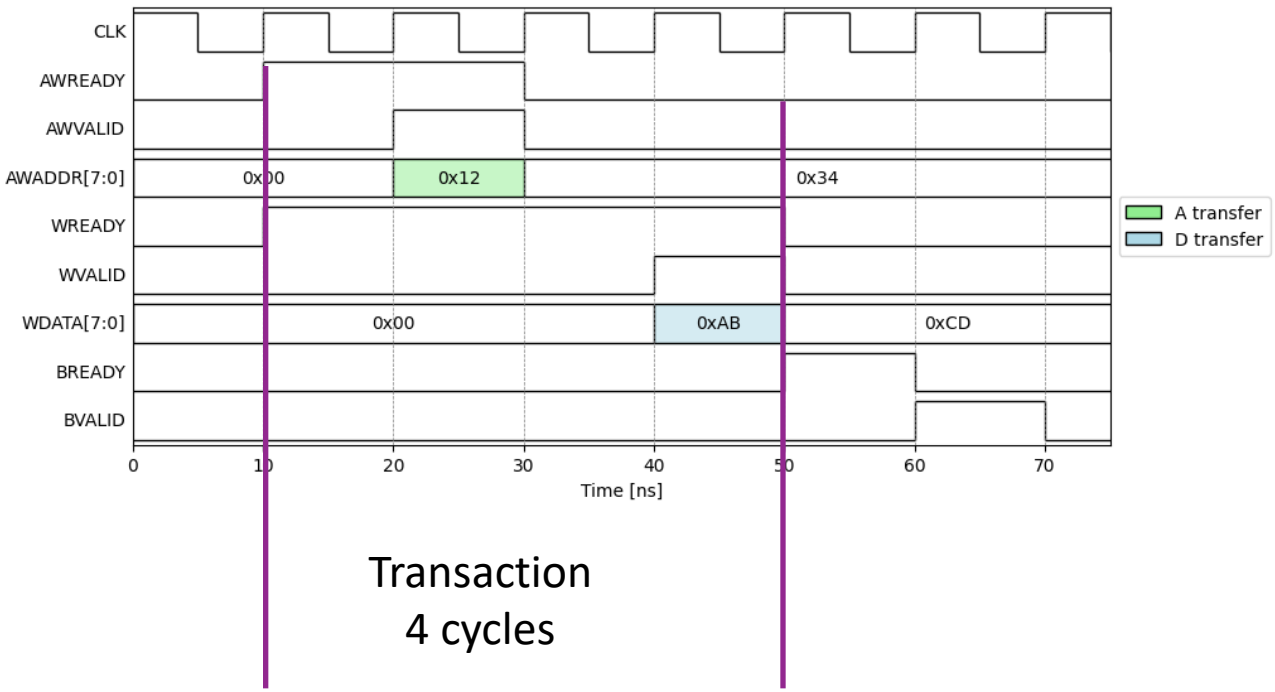
Outline

- FIFO Interfaces
- AXI4 Pure Streaming Interface
- Limitations of Pure Streaming with Vitis
- AXI4 Packetized Streaming

FIFO Motivation

- To motivate FIFOs, we review two problems we have encounter
- Problem 1: Low throughput on transfers with handshakes
- Problem 2: Idle time on systems with polling

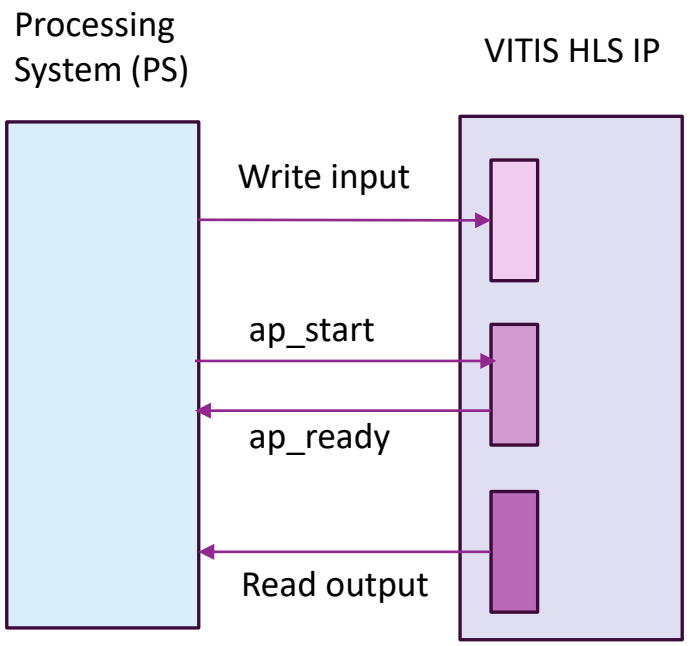
Problem 1: Handshaking Overhead



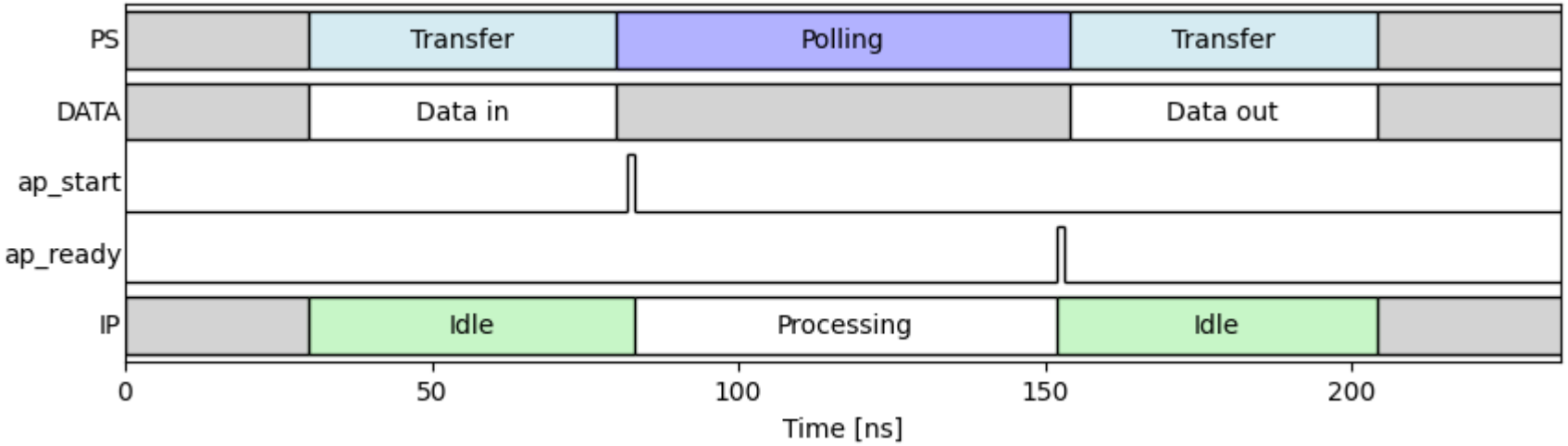
- Last Unit: Transfers with handshake
  - Slave asserts READY
  - Master assert VALID
- AXI4-Lite
  - Each read or write takes 2 to 4 cycles
- Provides:
  - Guaranteed delivery
  - Flow control
- But adds overhead

Problem 2: Polling-Based Execution

- Execution model in previous unit:
  - PS writes input data
  - PS sets ap_start
  - IP processes data
  - PS poll ap_ready until complete
  - PS reads output



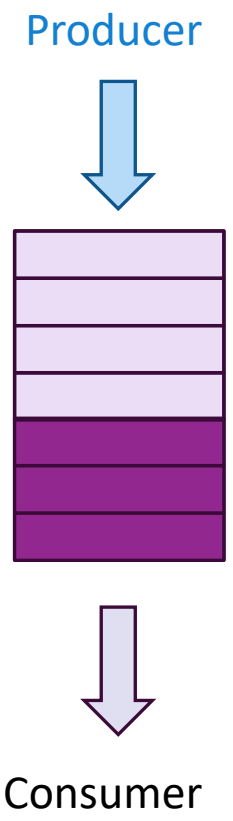
Problem 2: Time Wastage



- Several overhead times on both the PS and IP
  - Transfer: PS spends time transferring data in and out of IP
  - Polling: PS must poll while IP is processing
  - Idle: IP is idle during transfer and wait for control

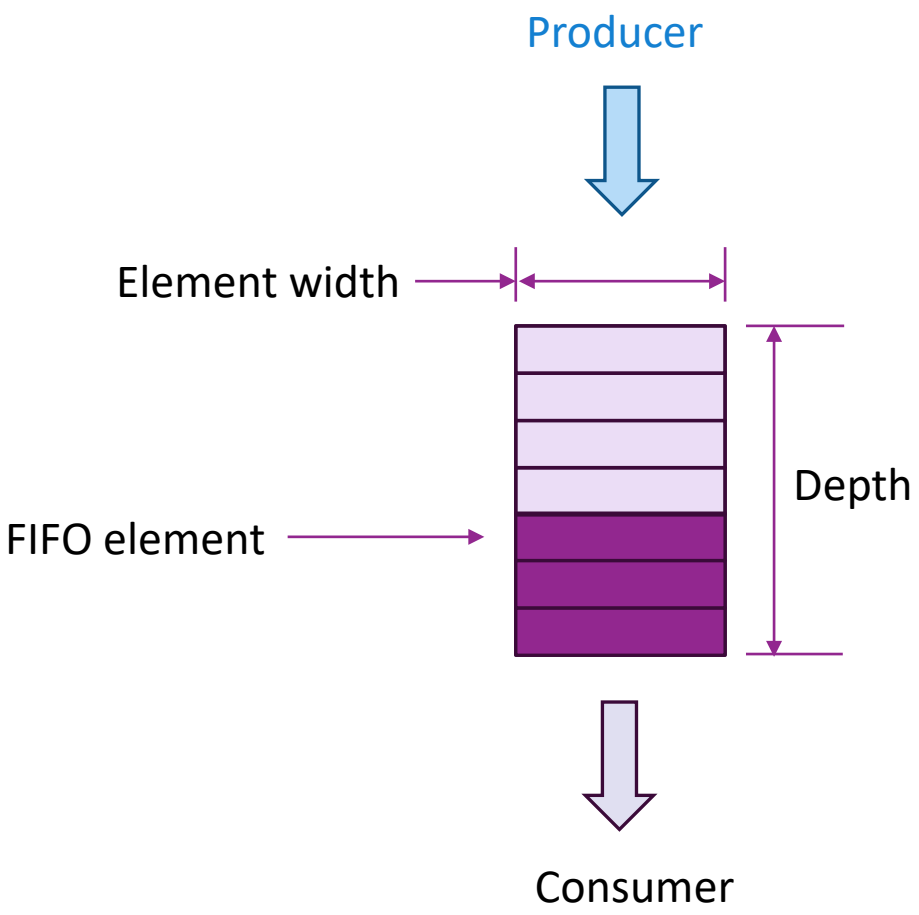
FIFO

- FIFO (First-In First-Out) hardware queue
  - Stores data from a producer in order of arrival
  - Returns data to a consumer in same order
- Decouples producer and consumer timing
  - Each side can run at their own pace
- Built-in flow control
  - Producer checks if FIFO is full
  - Consumer checks if FIFO is not empty
- Absorbs bursts of data
- Enables streaming behavior



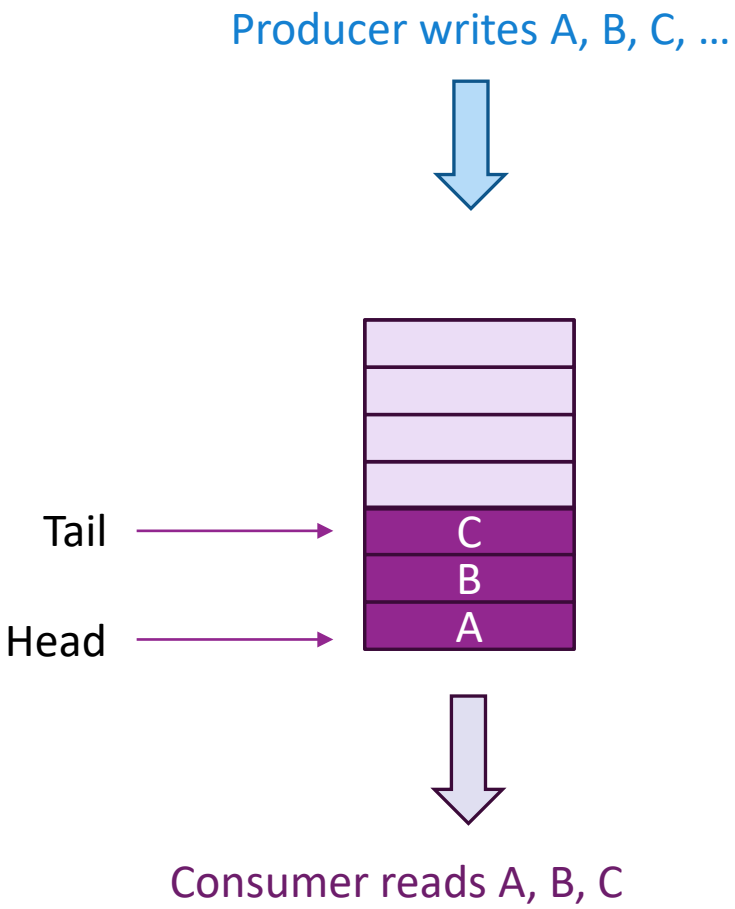
FIFO Terminology

- Producer: Module that writes to FIFO
  - Each write adds an element
- Consumer: Modules that reads from FIFO
  - Each read removes an element
- Element: Logical item stored in the FIFO
- Element width: Size of each element
  - Typically, in bytes or bits
- Depth: Maximum number of elements



FIFO Order

- Suppose FIFO writes A, B, C
- Then:
  - A is at the head of the queue
  - C is at the tail
- Consumer reads from the head
- Will read in order A, B, C
- Same order as the write



Example Problem

A FIFO buffer stores commands with three elements, here `ap_int` represents an arbitrary bit-width integer type with `n` bits.

```
struct Command {
    ap_int<16> command_id;
    ap_int<32> operand1;
    ap_int<32> operand2;
};
```

a. What is the element width in bytes?
b. If the FIFO has a depth of 32 elements, what is the total size of the FIFO in bytes?

Solution:

a. Each command element is represented by the following widths:

- command_id: 16 bits = 2 bytes
- operand1: 32 bits = 4 bytes
- operand2: 32 bits = 4 bytes

Therefore, the total element width is:

$$2 + 4 + 4 = 10 \text{ bytes}$$

b. If the FIFO has a depth of 32 elements, the total size of the FIFO is $32 \times 10 = 320$ bytes

Example Problem

Question 2. FIFO Timing

A producer writes data elements A, B, and C into a FIFO buffer at clock cycles 0, 1, and 6, respectively. Data can be read from the FIFO on the clock cycle immediately after it is written (for example, data element A can be read starting at clock cycle 1). Once a consumer reads an element, it takes 2 clock cycles to process it before the consumer can read the next element from the FIFO. Assuming the consumer attempts to read from the FIFO as soon as possible and starts at clock cycle 1, at which clock cycles will the consumer read:

a. Data element A
b. Data element B
c. Data element C

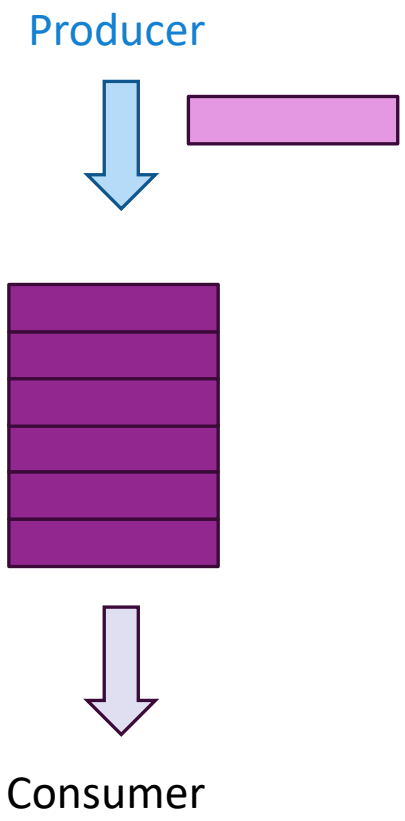
Solution

Cycle	Producer Action	FIFO Contents	Consumer Action
0	Write A	[A]	Idle
1	Write B	[B]	Read A (start processing A)
2	—	[B]	Processing A
3	—	[]	Read B (start processing B)
4	—	[]	Processing B
5	—	[]	Idle (ready to read)
6	Write C	[C]	Idle
7	—	[]	Read C (start processing C)

a. A is read at cycle 1.
b. B is read at cycle 3.
c. C is read at cycle 7.

FIFO Overflow

- Suppose producer tries to write an element when FIFO is full
- Two possible behaviors
- Blocking write
  - FIFO signal not READY
  - Producer waits
  - Default behavior in most hardware interfaces (including AXI-Stream)
  - Also called stall, backpressure or flow control
- Drop
  - Drops some packet in queues
  - Head, tail, or random
  - Common in network routers



Example Problem

Question 3. FIFO Write Stall

A producer generates data elements A, B, C, and D. The **earliest** clock cycles at which each element becomes available for writing into a FIFO are:

- A at cycle 0
- B at cycle 1
- C at cycle 2
- D at cycle 3

The FIFO has a capacity of **2 elements**. A write can occur only if the FIFO is not full. If the FIFO is full when an element becomes available, the producer must **stall** and retry the write on each subsequent cycle until it succeeds.

A consumer reads one element at a time from the FIFO. If data is written on cycle *N*, it is available to be read and removed from the FIFO on cycle *N+1*, and will process the element until cycle *N+3*. Thus, the next read is possible at cycle *N+4*.

For each element, determine the clock cycle at which the write **actually succeeds** (i.e., when the element is stored in the FIFO) and when the processing is complete.

Solution

Cycle	Producer Action	FIFO Contents	Consumer Action
0	A becomes available → write A	[A]	Idle
1	B becomes available → write B	[B]	Read A
2	C becomes available → write C	[C, B] (full)	Processing A
3	D becomes available → **stall** (FIFO full)	[C, B]	Processing A
4	D retries → **write D** (space freed)	[D, C]	Read B
5- 6	—	[D, C]	Processing B
7	—	[D]	Read C
8- 9	—	[D]	Processing C
10	—	[]	Read D
11- 12	—	[]	Processing D

Producer and Consumer Rate

- Producer rate = average number of elements to write per unit time
  - Time unit can be clock cycle, seconds, etc
  - If producer is sending jobs, producer rate = workload rate requirement

- Consumer rate = maximum number of reads per unit time
  - If consumer processes element, consumer rate = maximum work throughput

- Load factor: $\rho = \frac{\text{Producer rate}}{\text{Consumer rate}}$

Load Factor and Stability

Condition	Queueing theory term	Hardware term	Implication
$\rho < 1$	Stable	Under-utilized; headroom available	FIFO will always eventually drain. No necessary drops / stalls. But short-terms drops / stalls may be possible
$\rho = 1$	Critical	Fully utilized	Stalls / drops may or may not occur.
$\rho > 1$	Unstable	Overloaded, over-subscribed	With backpressure, producer will have infinite backlog. Without backpressure, some elements will be dropped

Sample Problem

Question 4. CV Accelerator Throughput and FIFO

A computer vision accelerator processes a stream of video frames. Each frame requires **5 ms** of base processing time. On **10%** of the frames, an object of interest is detected. For these frames, the accelerator requires an additional **100 ms** of processing time (for a total of 105 ms on those frames). The accelerator processes frames one at a time.

Assume frames arrive continuously and the accelerator must keep up with the input stream without the input FIFO growing without bound.

a. What is the maximum frame rate (in frames per second) that the accelerator can sustain without falling behind?
b. Now suppose the accelerator is preceded by an input FIFO that buffers frames before they are processed. The probability that a frame requires the extra 100 ms of processing is still 10%, but the *pattern* of when these expensive frames occur can differ. Rank the following three patterns in terms of how large the input FIFO must be to avoid overflow (from **smallest** required FIFO size to **largest**):
    i. Objects of interest arrive uniformly, e.g., on frames 0, 10, 20, 30, ...
    ii. Objects of interest arrive in bursts, e.g., on frames 0, 1, 2, then 30, 31, 32, ...
    iii. Objects of interest arrive randomly, e.g., frames 15, 21, 27, ... (but 1/10 on average).

Solution

a. On 90% of frames, the cost is 5 ms. On 10% of frames, the cost is 5 + 100 = 105 ms. The average processing time per frame is:

   0.9 × 5 ms + 0.1 × 105 ms = 4.5 ms + 10.5 ms = 15 ms per frame.

   The maximum sustainable frame rate is the inverse of the average processing time:

   FPS_{max} = 1 / 0.015 s ≈ 66.7 frames per second.

b. All three patterns have the same *average* fraction of expensive frames (10%), but they differ in how *bursty* the expensive work is. The more bursty the arrivals, the larger the FIFO must be to absorb temporary backlogs.

   - **(i) Uniform:** Expensive frames are perfectly spaced (every 10th frame). This produces the smoothest workload and the smallest buildup in the FIFO. This requires the **smallest** FIFO.
   - **(iii) Random:** On average, this behaves like the uniform case, but randomness can create occasional short bursts of expensive frames. The FIFO must be larger than in the uniform case to handle these bursts, but extreme clustering is relatively rare.
   - **(ii) Bursty:** Expensive frames arrive in tight clusters (e.g., 0, 1, 2 and then 30, 31, 32, ...). During a burst, the accelerator falls behind significantly, and the FIFO must be large enough to hold the backlog created by several expensive frames in a row. This requires the **largest** FIFO.

   Therefore, ranking from smallest to largest required FIFO size: **(i) Uniform < (iii) Random < (ii) Bursty**.

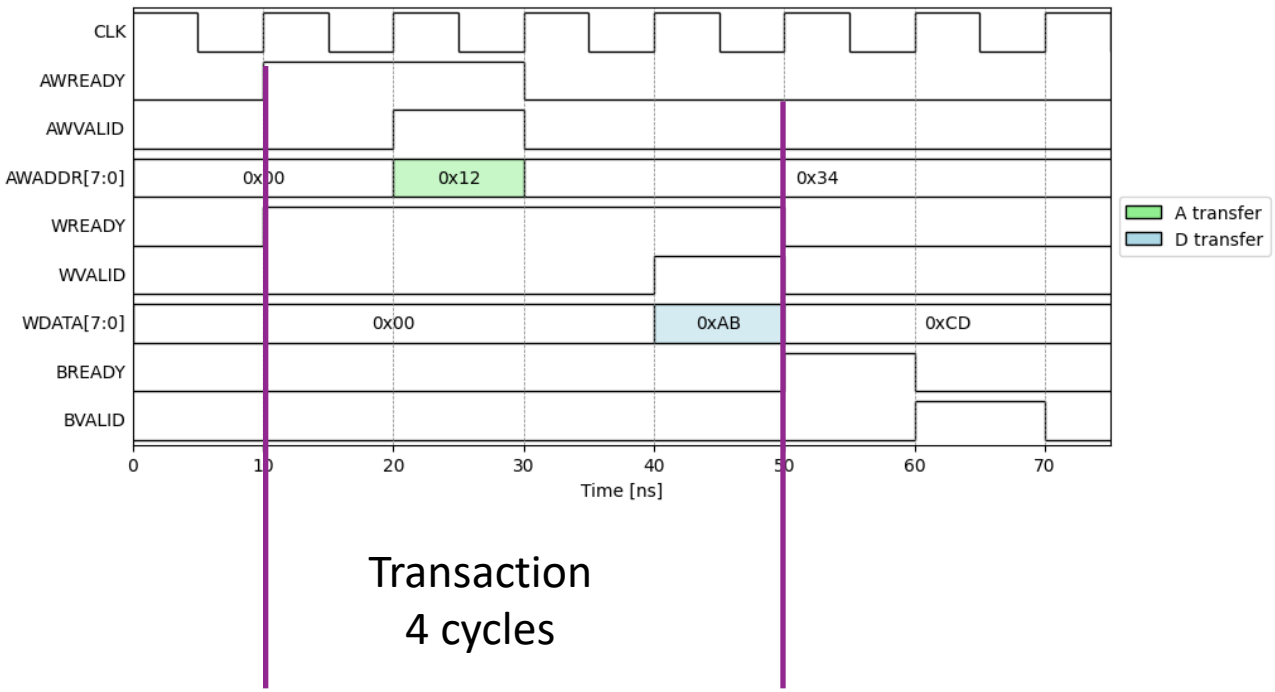
Queueing Theory

- Models arrivals and departures probabilistically
- Used widely in networking research
  - Determine probabilities of buffer overflow
  - Optimal routing and drop policies
  - Also used in network on chip
- But requires mathematics
  - Markov chains, probability
- Take NYU's networking theory class if interested

Outline

- FIFO Interfaces
- AXI4 Pure Streaming Interface
- Limitations of Pure Streaming with Vitis
- AXI4 Packetized Streaming

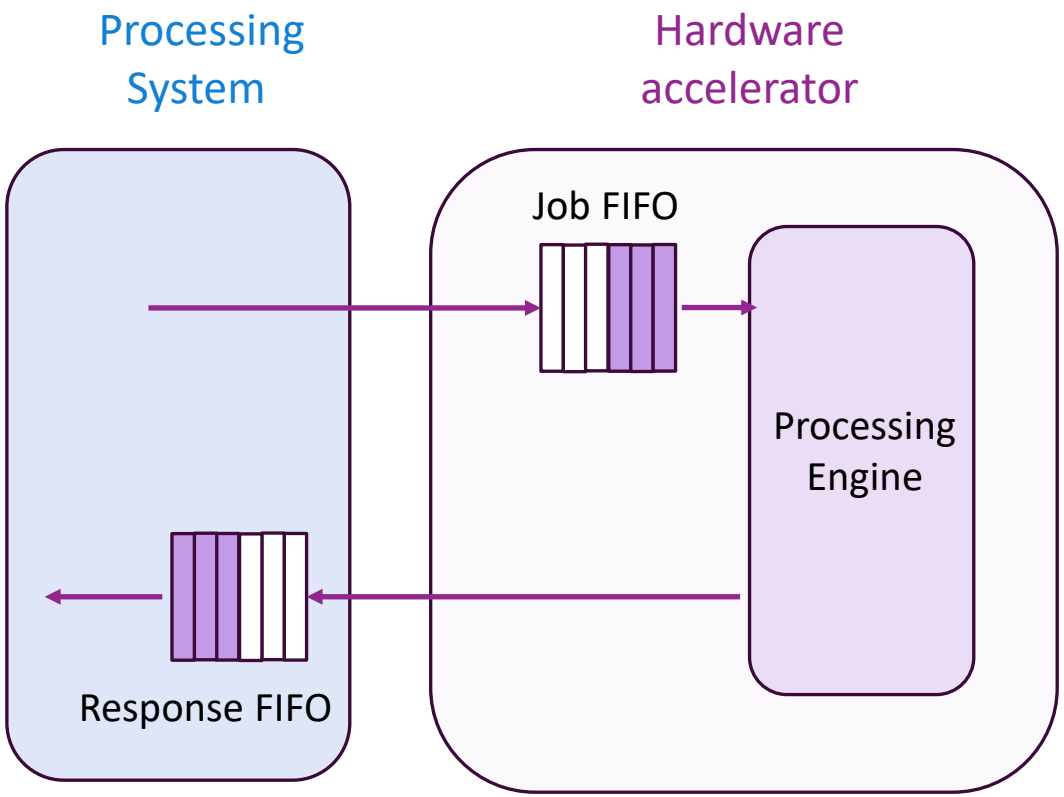
Problem 1: Handshaking Overhead



- Last Unit: Transfers with handshake
  - Slave asserts READY
  - Master assert VALID
- AXI4-Lite
  - Each read or write takes 2 to 4 cycles
- Provides:
  - Guaranteed delivery
  - Flow control
- But adds overhead

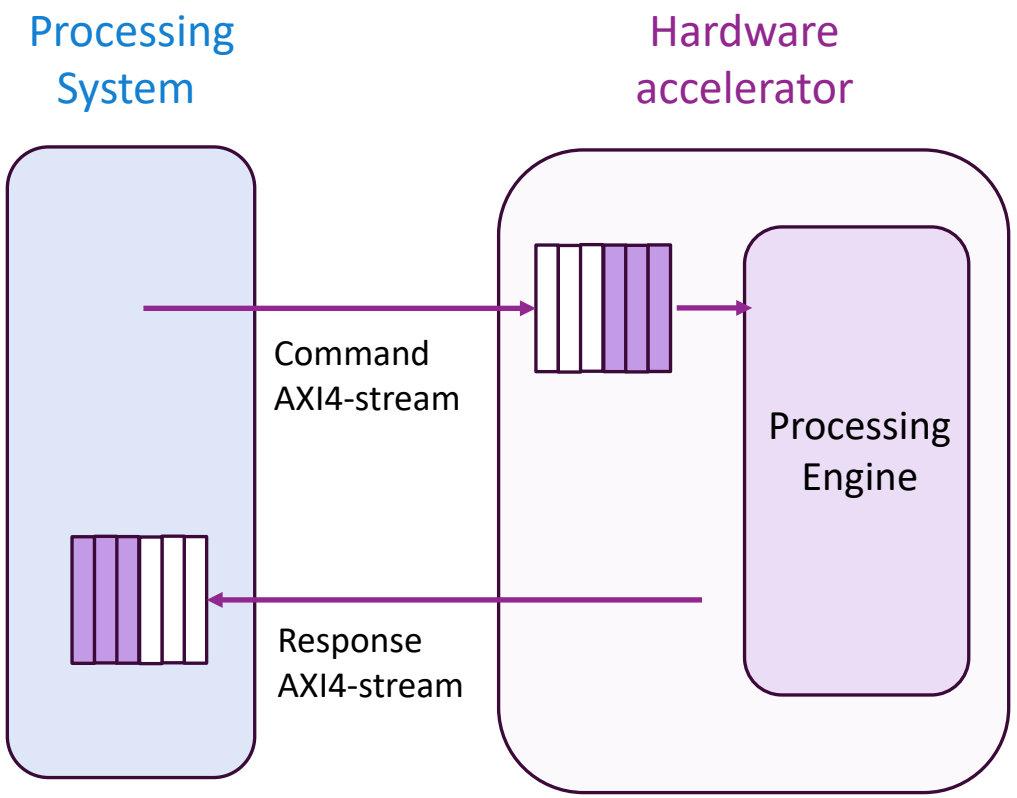
Solution: Streaming FIFO Interface

- Consider a PS sending jobs to accelerator
- Use FIFOs for jobs and responses
- Decouples for timing
  - Transfer of data of the PS
  - Processing of data in the HW
- Enables efficient burst transfers
  - Many elements can be transferred at once
  - Reduces handshaking overhead
- Both PS and HW run at their own pace
- If some jobs need a lot of time
  - Job FIFO buffers the jobs to process

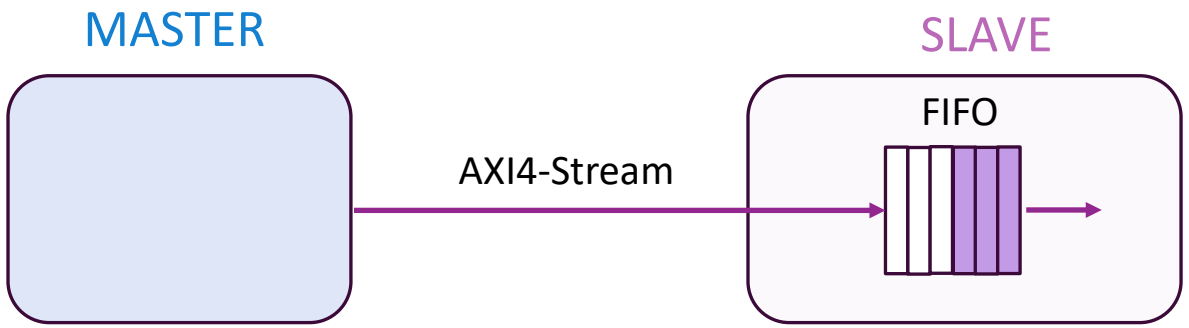


AXI4-Streaming Protocol

- Industry standard streaming protocol
  - Available as the interface to many Xilinx IP blocks
- Pre-built in Vitis HLS
  - All FIFO management is done for you
- Part of the AXI4 family:
  - AXI4-Lite: Covered last unit
  - AXI4-Stream: This unit
  - AXI4-Memory mapped: Later
- Provides a simple interface with DATA, READY, VALID
- Additional capabilities (covered later in this unit):
  - Packetization
  - Addressing



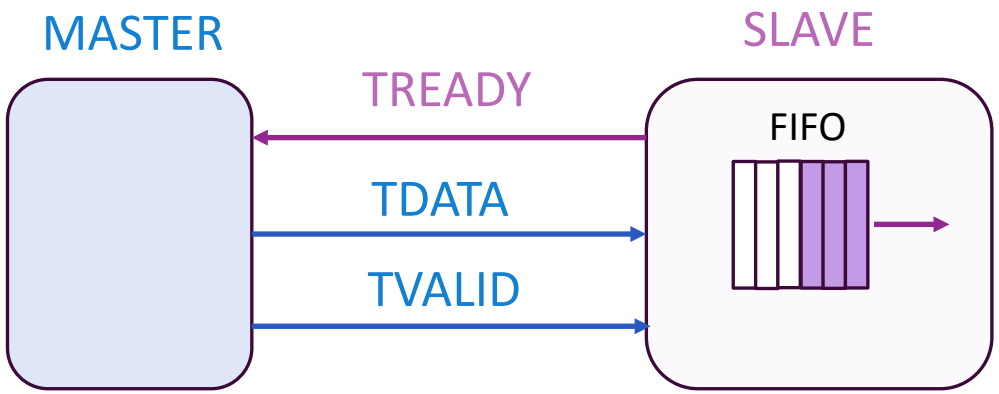
AXI4-Stream Terminology



- Each AXI4-Stream is uni-directional
- Master is the producer / sender
  - Could be PS sending jobs or HW accelerator returning responses
- Slave is the consumer / receiver
  - Could be PS receiving response or HW accelerator receiving jobs
- FIFO is typically realized on the

AXI4-Stream Protocol

- Slave asserts TREADY on any cycle it can accept data
  - Typically, when FIFO has space
- On any cycle has data to transfer, master:
  - Places data on TDATA
  - Asserts TVALID
- If TREADY=TVALID=1, there is a transfer
  - Otherwise, master can re-assert TVALID on next cycle
- Slave processes FIFO at its own pace
  - Backpressure to master via TREADY
- Can transfer up to data element



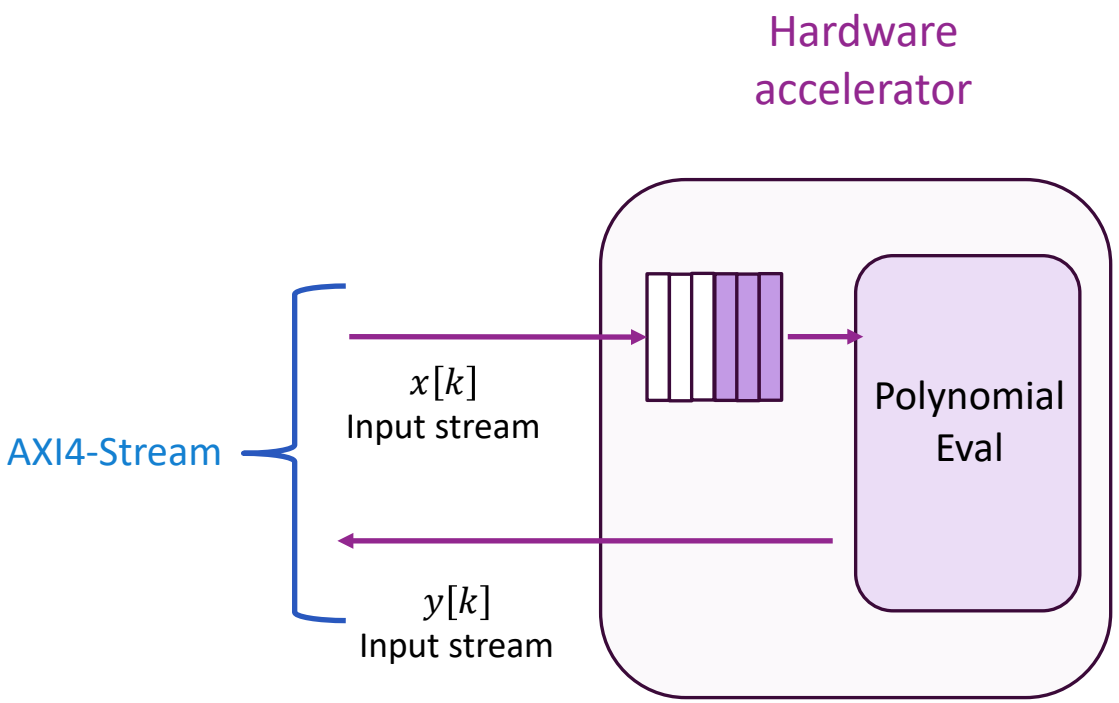
Example Timing

Cycle	Master action	TVALID	TREADY	FIFO	Slave action	TREADY for next cycle
0	Transfer A	1	1	[A]	-	1
1	Transfer B	1	1	[B]	Read A	1
2	Transfer C	1	1	[C,B]	Process A	0
3	D stalls	1	0	[C]	Read B	1
4	Transfer D	1	1	[D,C]	Process B	0
5	E stalls	1	0	[D]	Read C	1
6	Transfer E	1	1	[E,D]	Process C	0

- Master always has data
  - TVALID=1 always
- Slave takes 2 cycles to:
  - Read data
  - Process data
- FIFO depth = 2

Example: Streaming Squared Average

- Given:
  - Input stream $x[k]$
- Compute output stream:
  - $y[k] = \frac{1}{L}(x[k]^2 + \ \ldots + x[k-L+1]^2)$
  - L is a fixed window size
- Models an average power
- Use AXI4-Stream for $x[k]$ and $y[k]$



Vitis HLS Module Definition

```
void avgfilt(
    hls::stream<float>& in_stream,
    hls::stream<float>& out_stream)
{

#pragma HLS INTERFACE axis port=in_stream
#pragma HLS INTERFACE axis port=out_stream
#pragma HLS INTERFACE ap_ctrl_none port=return
```

Input and output streams

Declares that variables are AXI4-Stream
Does not state if master or slave.

Declares that module has no explicit start stop

Vitis HLS Module Body

```
    while (1) {

#pragma HLS PIPELINE II=1
        // Read value
        float xi = in_stream.read();
        float xsq = xi*xi;

        // Output value
        float yi = (xsq + xsq0 + xsq1)*inv_win_size;
        out_stream.write(yi);

        // Update delay line
        xsq1 = xsq0;
        xsq0 = xsq;

    }
```

Reads input data

Computes average of current + past 2 values.

Writes output

Outputs past values

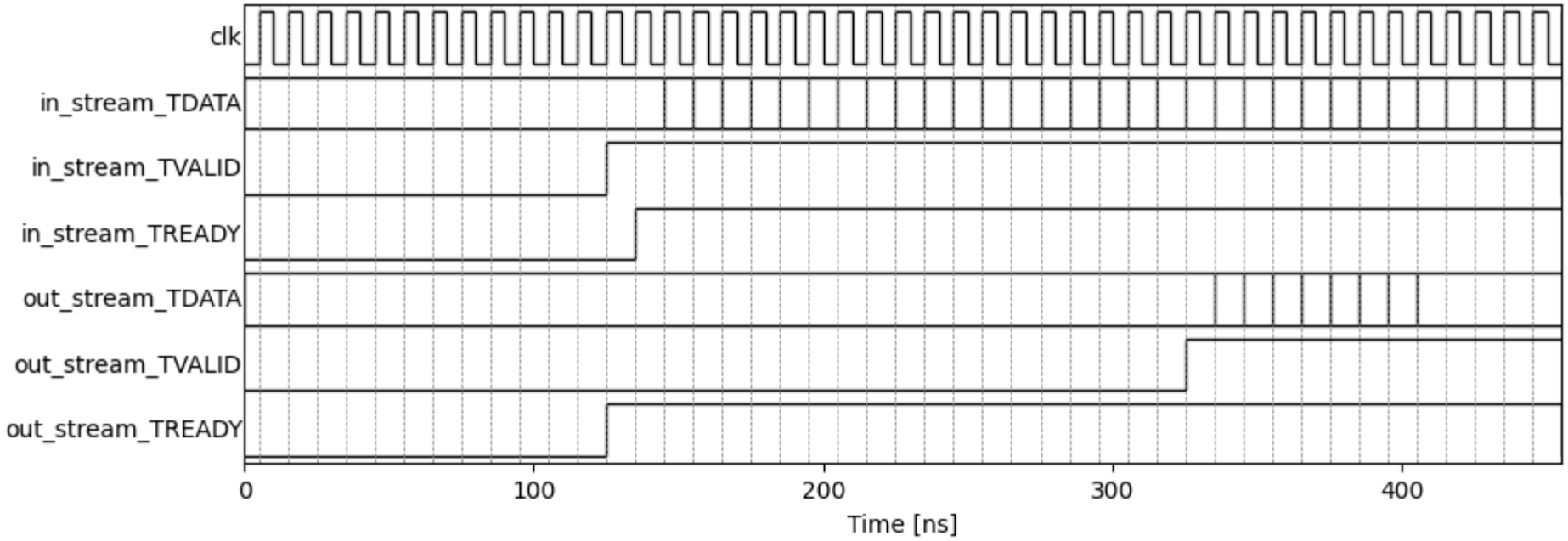
In-Class Demo

We will perform

- ❑ C simulation
- ❑ C synthesis
- ❑ RTL co-simulation
- ❑ VCD extraction

```
# ==============================================================
# Vitis HLS TCL script for the average filter (avgfilt) demo
# ==============================================================

# Create or reset the project
open_project hls_component -reset

# Set the top-level function
set_top avgfilt

# --------------------------------------------------------------
# Add design and testbench files
# --------------------------------------------------------------
# Adjust paths if your directory structure differs
add_files avgfilt.cpp
add_files -tb tb_avgfilt.cpp

# --------------------------------------------------------------
# Create solution and configure hardware part + clock
# --------------------------------------------------------------
open_solution "solution1" -reset

# FPGA part (Pynq-Z2)
set_part {xc7z020clg400-1}
```

Simulation Results



Outline

- ❑ FIFO Interfaces
- ❑ AXI4 Pure Streaming Interface
- ❑ Limitations of Pure Streaming with Vitis
- ❑ AXI4 Packetized Streaming

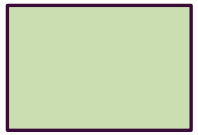
Limitations of Pure Streaming

- Problem 1: Many operations fundamentally operate in blocks
  - Pure streaming interface does not natively have a block boundary
- Problem 2: Hard to synchronize control and data
- Problem 3: Vitis is not well-equipped to simulate continuous kernels
  - Vitis has a fundamentally flawed execution model

Problem 1: Block-Based Operations

- Many problems naturally work on blocks
- Need to know boundaries
- Ex: Image processing
  - Each input is a image
  - Images have a first and last element
  - Scanned in some particular order (e.g. row-wise)
- Ex 2: Matrix multiply
  - Module needs two matrices
  - Each has to be read in some order
- Pure streaming is not natively good for these operations

Individual elements



Blocks of elements

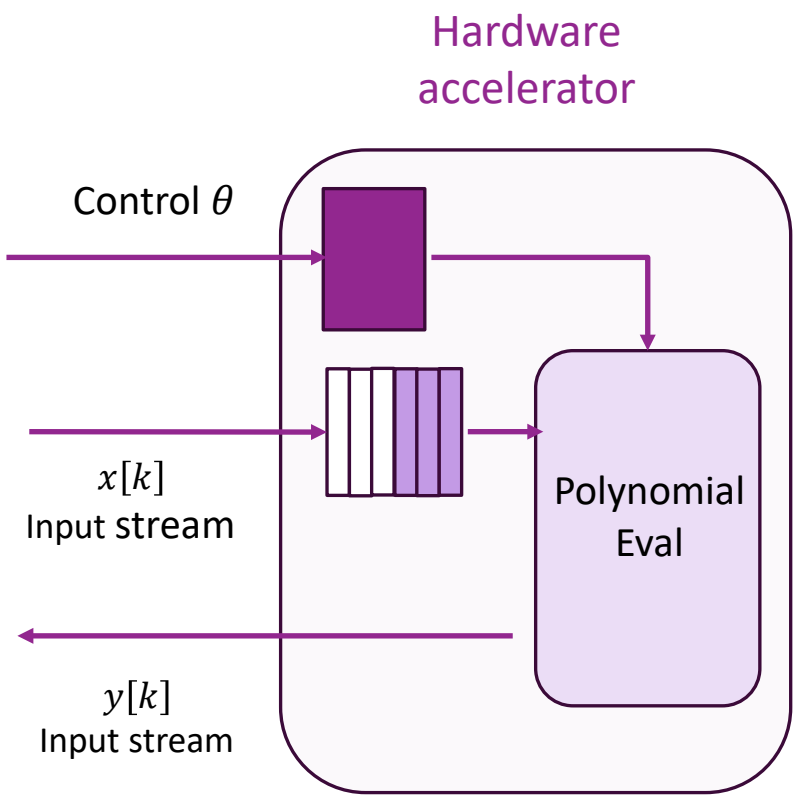
Problem 2: Control-Data Synchronization

- This problem is more subtle
- Consider a parametrized streaming function:

$$y[k] = f(x[k], \theta)$$

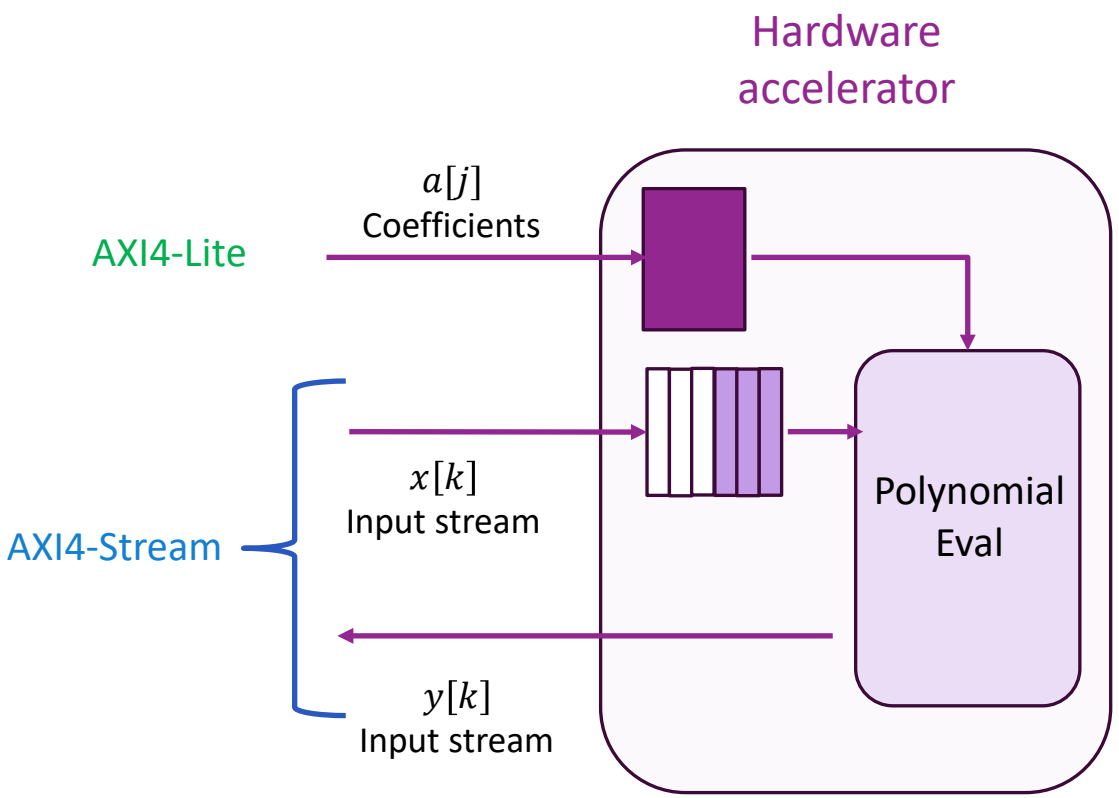
  - θ : some parameter that changes slowly

- For example, streaming polynomial:
  - $y[k] = a_0 + a_1 x[k] + a_2 x[k]^2$
  - $\theta = (a_0, a_1, a_2)$



Solution: Mix AXI4-Lite and AXI4-Stream?

- Given:
  - Input stream $x[k]$
  - Polynomial coefficients $a[j]$ (change slowly)
- Compute output stream:
  - $y[k] = a[0] + a[1]x + \ldots + a[d]x^d$
- Set coefficients via AXI4-Lite
  - Assume they change slowly
  - Do not need fast read / write
  - But benefit from continuous access
- Use AXI4-Stream for $x[k]$ and $y[k]$



A Potential HLS Module Definition

```
void stream_fun(
    hls::stream<float> &in_stream,
    hls::stream<float> &out_stream,
    float a0,
    float a1,
    float a2
) {
// AXI4-Stream interfaces
#pragma HLS INTERFACE axis port=in_stream
#pragma HLS INTERFACE axis port=out_stream

// Set AXI-Stream FIFO depths
#pragma HLS STREAM variable=in_stream  depth=4
#pragma HLS STREAM variable=out_stream depth=4

// AXI4-Lite control interface for parameters and start signal
#pragma HLS INTERFACE s_axilite port=a0     bundle=CTRL
#pragma HLS INTERFACE s_axilite port=a1     bundle=CTRL
#pragma HLS INTERFACE s_axilite port=a2     bundle=CTRL
#pragma HLS INTERFACE s_axilite port=return bundle=CTRL
```

Input and output streams

Coefficients

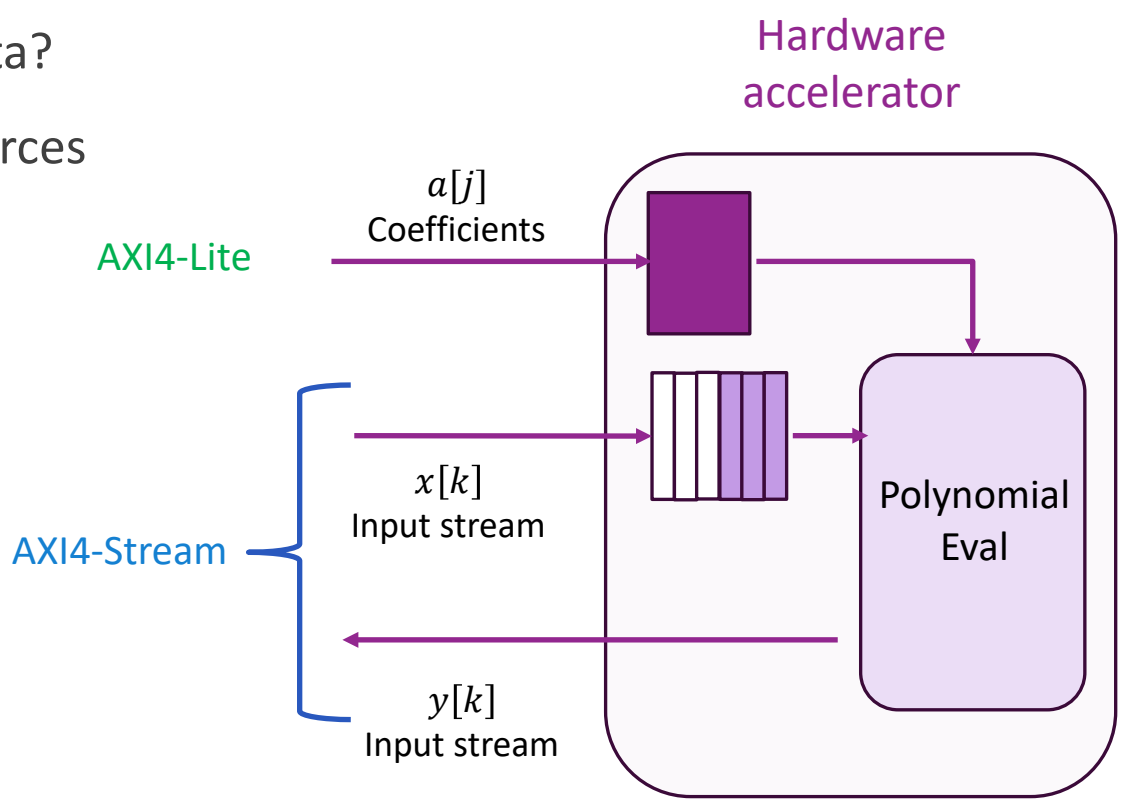
Declares that variables are AXI4-Stream
Does not state if master or slave.
Direction will be inferred

AXI4-Stream FIFO depth

Declares that coefficients are AXI4-Lite

Problem Mixing AXI4-Lite and AXI4-Stream

- Problem: How do we synchronize control and data?
- Control and data are typically fed by different sources
- Exact timing of control is indeterminate.
- Bad: *"Change control around now"*
- Good:
  - Index samples by k
  - *"Apply this control precisely on sample $k = 1000$"*
- This synchronization needs additional logic



Problem 3: Vitis' Flawed Execution Model

I need to accelerate this step

Software developer

```
// Compute x0 -> x1 -> x2
easy_function1(x0, x1);
easy_function2(x1, x2);

// Compute x2 -> x3
hard_function(x2, x3);

// Compute x3 -> x4
easy_function3(x3, x4);
```

XILINX VITIS

Vitis generated RTL equivalent code

- Drop-in replacement for SW call
- Called with sequential manner

```
module hard_function(...);
    ...
endmodule
```

Vitis Execution Model Flaws

Item	Vitis Assumption	Proper hardware model
Execution paradigm	Module is “called” like a C function	Modules operate continuously and respond to events
Interface	C-like call	Hardware channels e.g., FIFOs and memory-mapped interfaces
Concurrent behavior	Obscured	Made explicit

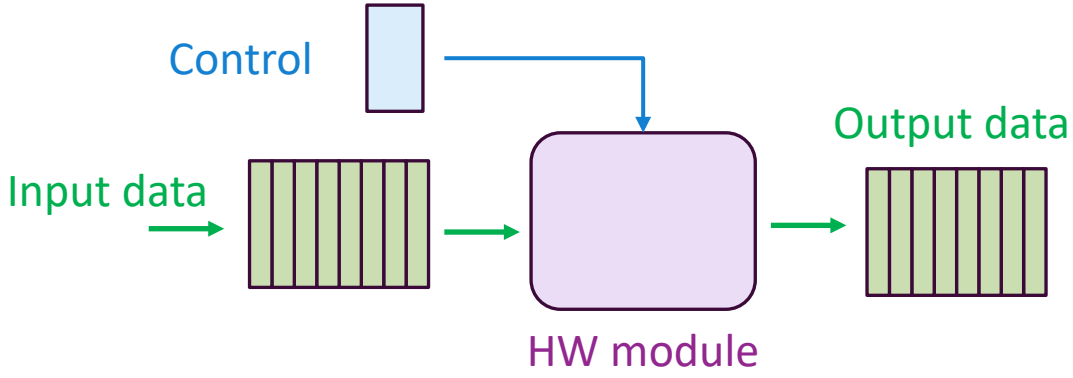
Hardware is not software

Outline

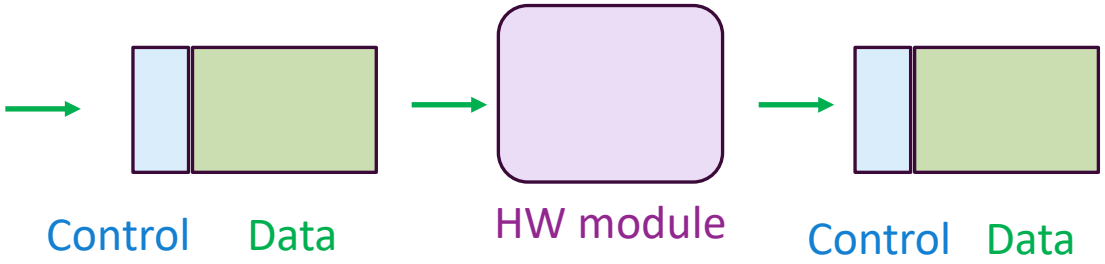
- FIFO Interfaces
- AXI4 Pure Streaming Interface
- Limitations of Pure Streaming with Vitis
- AXI4 Packetized Streaming

Pure vs. Packetized Streaming

- Pure streaming
  - Module processes one data element at a time
  - Ex: $y[k] = f(x[k], \theta)$
  - Control (e.g., θ) set in a separate path

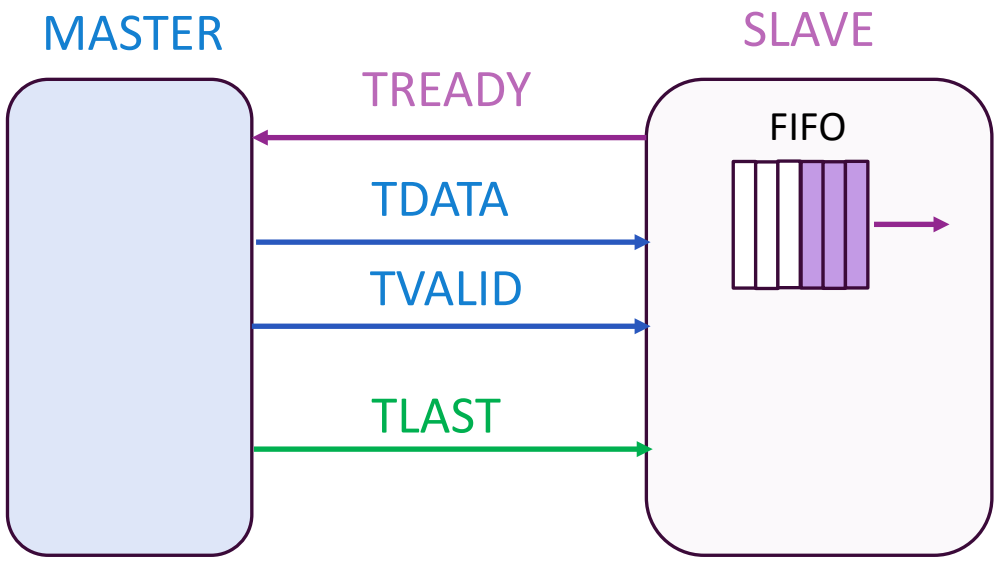


- Packet streaming
  - Module processes data in blocks or packets
  - Control may be sent with packet



AXI4-Stream Protocol with Packetization

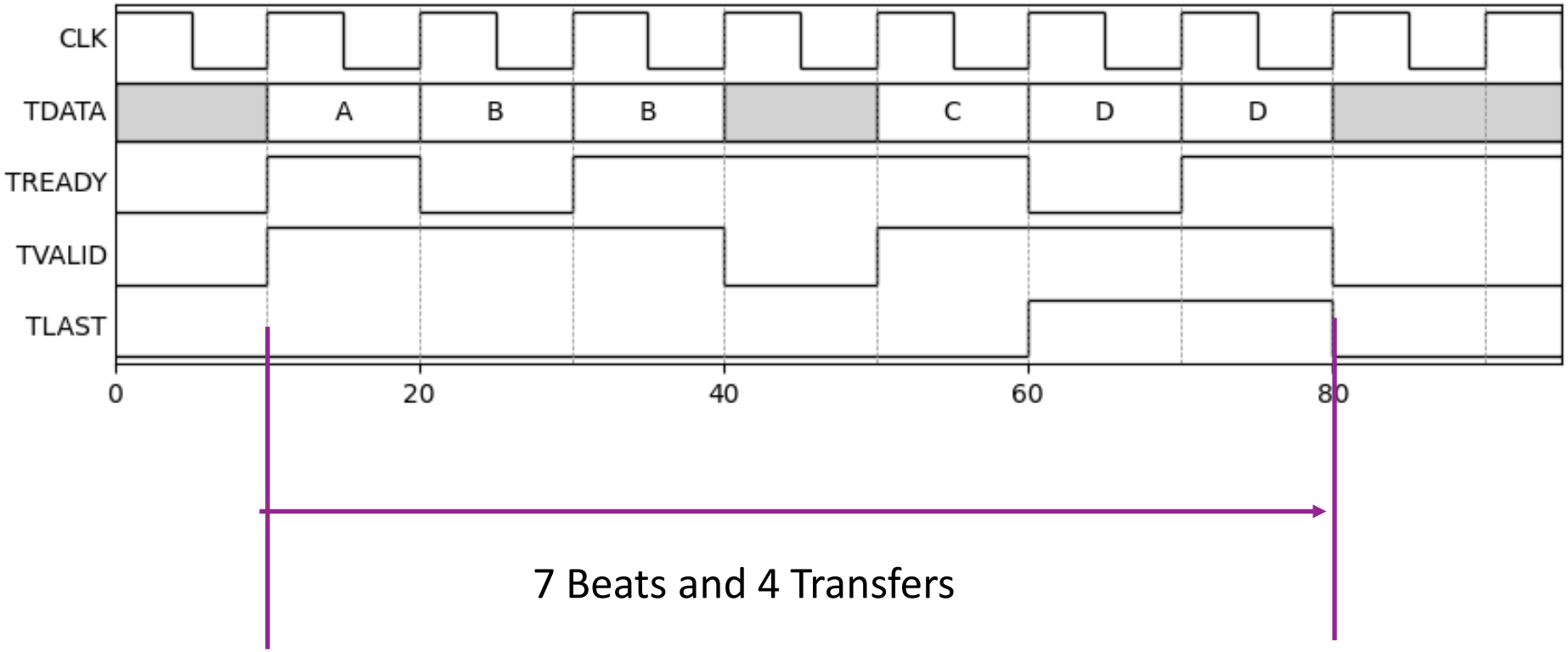
- Add a TLAST line
- TLAST = 1 on the last transfer
- Indicates the end of packet
- Packets are transferred in bursts
  - Begins with first TVALID = 1
  - Ends TLAST=1 and successful transfer
- Cycles in a burst are called beats
  - Transfer beat (TVALID=TREADY=1)
  - Idle beat (TVALID=0)
  - Stall beat (TVALID=1, TREADY=0)
  - Last beat (TVALID=TREADY=TLAST=1)



Example Burst Timing

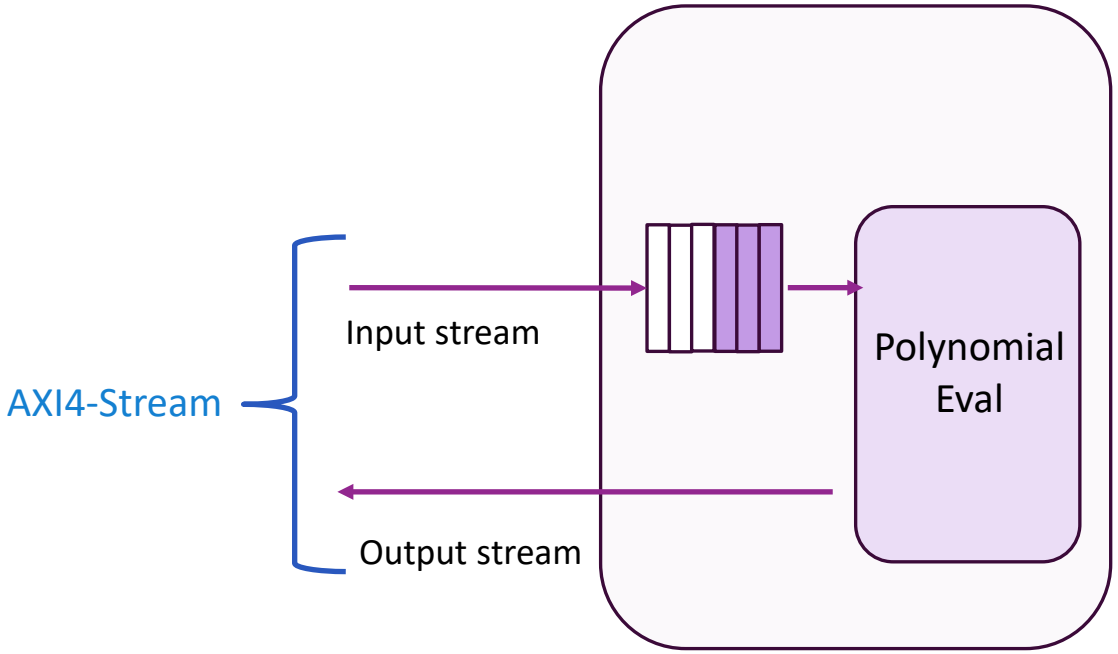
TDATA	TVALID	TREADY	TLAST	Beat Type
A	1	1	0	A transfers
B	1	0	0	B stalls (Slave not ready)
B	1	1	0	B transfers
-	0	1	0	Idle (Master has no data)
C	1	1	0	C transfers
D	1	0	1	D stalls
D	1	1	1	D transfers with end of burst

Example Burst Timing Illustrated

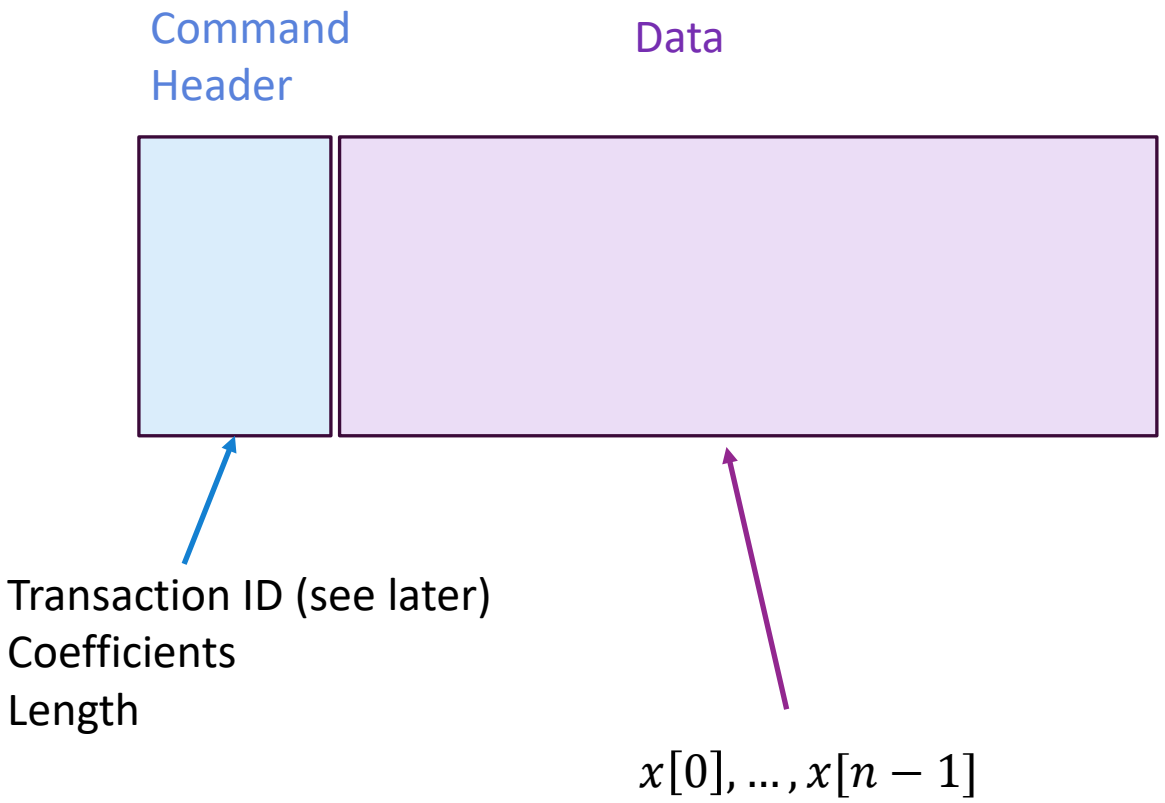


Example: Block Polynomial Accelerator

- Given:
  - Input stream $x[k]$
  - Polynomial coefficients $a[j]$ (change slowly)
- Compute output stream:
  - $y[k] = a[0] + a[1]x + \ldots + a[d]x^d$
- Idea: In each burst transfer
  - Coefficients $a[0], \ldots, a[d]$
  - Length n
  - Data $x[0], \ldots, x[n-1]$

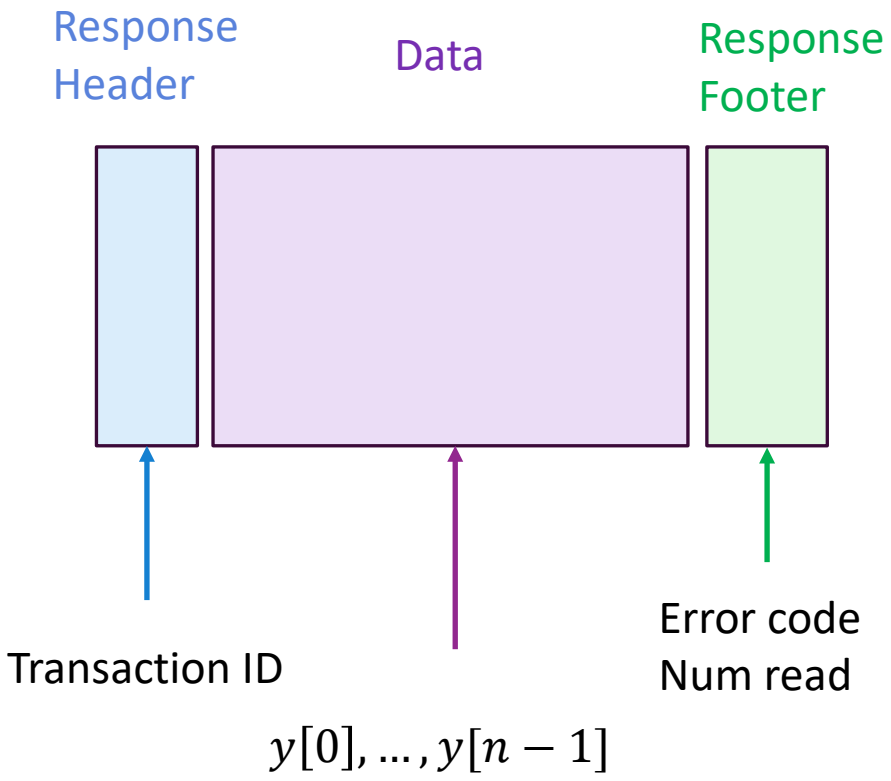


Command Packet Structure




```
class CmdHdr {
public:

    ap_int<16> trans_id; // Transaction ID
    float a0; // Coefficient a0
    float a1; // Coefficient a1
    float a2; // Coefficient a2
    ap_uint<32> n; // Number of data points
```

Response Packet Structure

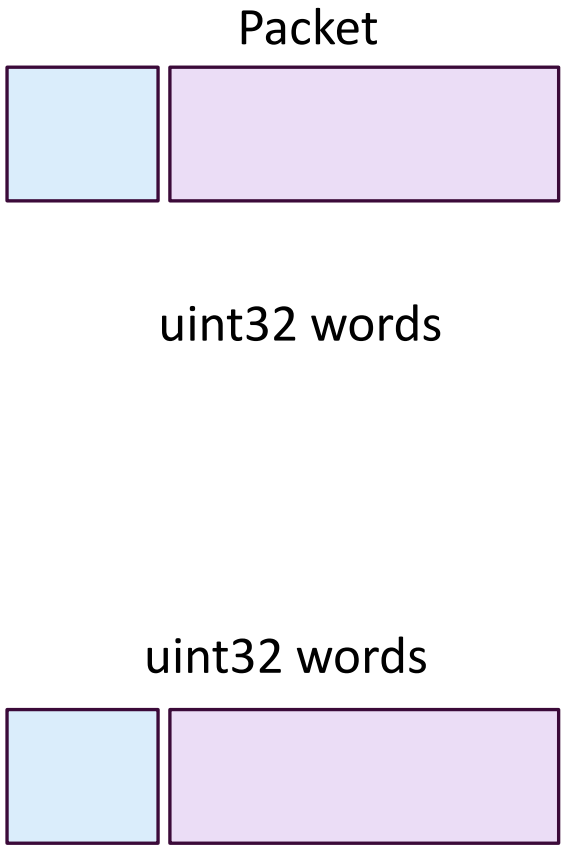



```
class RespHdr {
public:

    ap_int<16> trans_id; // Transaction ID
```

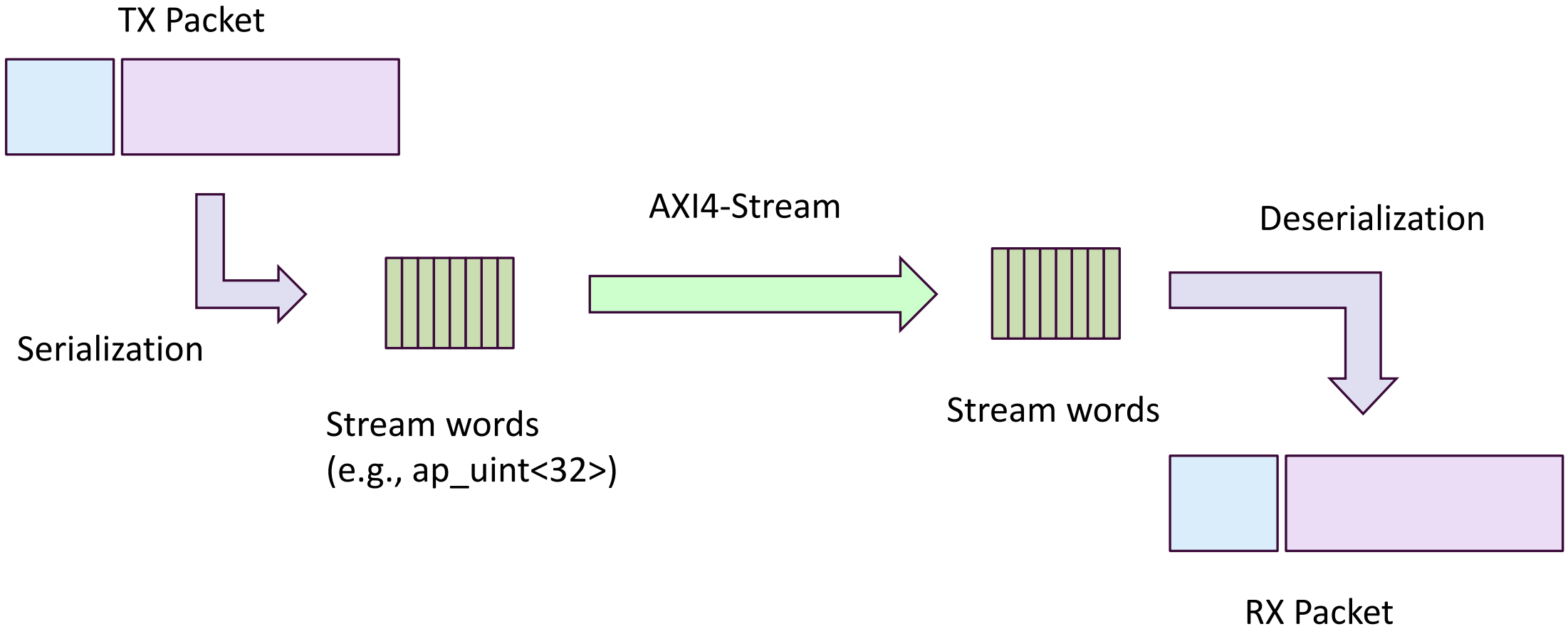
```
class RespFooter {
public:

    enum ErrCodes : unsigned int {
        NO_ERR = 0,
        SYNC_ERR = 1
    };

    ap_uint<32> nread; // Number of data points read before TLAST
    ap_uint<8> err_code; // Error Code
```

Serialization / Deserialization

- Packets have arbitrary:
  - Structure (often different fields)
  - Arbitrary, sometimes variable, length
- Hardware streams have specific bit widths
  - Ex: 32, 64, 128, …
  - Each stream element is a word
- Serialization: Converts packets to array of stream words
- Deserialization: Streams words to packets



Serialization / Deserialization Illustrated



Serialization / Deserialization in Hardware

- Python and C++ have widely-used serialization tools
- Ex: Python pickle
  - Can serialize an arbitrary class
  - Serializes complex nested structures
- Ex: Boost.serialization
  - Similar package in C++
- No standard library in hardware
- Often write custom serialization

```python
import pickle

# 1. The Python object to be saved (pickled)
data_to_save = {
    "name": "Jane Doe",
    "age": 30,
    "city": "Anytown",
    "is_active": True
}

# 2. Save the dictionary into a file
# Open a file in "write binary" ('wb') mode
with open('profile.pkl', 'wb') as file:
    # Use pickle.dump() to serialize the data into the file
    pickle.dump(data_to_save, file)
```

Example Problem

Question 8. Serializing Command Header

You are given a command header with the following data structure. Here `ap_uint` represents an unsigned integer with `n` bits, and `ap_fixed` represents a signed fixed-point value with `W` total bits and `I` integer bits (including the sign bit).

```
class CmdHdr {
    ap_uint<16> trans_id;   // Transaction ID

    // Rotation angles in Q12.10 format, where 1.0 corresponds to π radians
    ap_fixed<12,1> yaw, pitch, roll;

    ap_uint<16> nsamp;      // Number of samples to rotate
};
```

a. How many 32-bit words are required to store one instance of this header? Assume that no field may be split across word boundaries.
b. Complete the function below to serialize an instance of this header into an array of 32-bit words. There are multiple valid layouts; choose one that uses the minimum number of 32-bit words.

```
void serialize(const CmdHdr &cmd_hdr, ap_uint<32> words[/* fill in */]) {
    ...
}
```

Solution: Counting the Number of Words

Solution:

a. The bit-widths of the fields are:

- trans_id: 16 bits
- yaw: 12 bits
- pitch: 12 bits
- roll: 12 bits
- nsamp: 16 bits

Total width: $16 + 12 + 12 + 12 + 16 = 68$ bits.

Because no field may cross a 32-bit boundary, the minimum packing is:

- Word 0: trans_id (16) + yaw (12) = 28 bits
- Word 1: pitch (12) + roll (12) = 24 bits
- Word 2: nsamp (16)

Therefore, **3 words** are required.

Solution: Serialization Code

```
void serialize(const CmdHdr &cmd_hdr, ap_uint<32> words[3]) {
    // Word 0: trans_id | yaw
    ap_uint<32> w0 = 0;
    w0.range(15, 0)  = cmd_hdr.trans_id;
    w0.range(27, 16) = cmd_hdr.yaw.range(11, 0);
    words[0] = w0;

    // Word 1: pitch | roll
    ap_uint<32> w1 = 0;
    w1.range(11, 0)  = cmd_hdr.pitch.range(11, 0);
    w1.range(23, 12) = cmd_hdr.roll.range(11, 0);
    words[1] = w1;

    // Word 2: nsamp
    ap_uint<32> w2 = 0;
    w2.range(15, 0) = cmd_hdr.nsamp;
    words[2] = w2;
}
```

Vitis HLS Module Definition

```
void poly_block(
    hls::stream<stream_t>& cmd_stream,
    hls::stream<stream_t>& resp_stream)
{
#pragma HLS INTERFACE axis port=cmd_stream
#pragma HLS INTERFACE axis port=resp_stream
#pragma HLS INTERFACE ap_ctrl_none port=return
```

Input and output streams

Declares that variables are AXI4-Stream
Does not state if master or slave.

Declares that module has no explicit start stop

Vitis HLS Command and Response Header

```
CmdHdr cmd_hdr;

// Read command from input FIFO using stream_read
bool tlast = cmd_hdr.stream_read<stream_t>(cmd_stream);


// Write the response header with the echo-ed transaction ID
// TLAST is not set on the response header since there will
// be data words following the header
RespHdr resp_hdr;
resp_hdr.trans_id = cmd_hdr.trans_id;
resp_hdr.stream_write<stream_t>(resp_stream, false);
```

Deserialize command header

Write response header
Each transaction ID

Main Process Loop

```
// Perform calculation
float a0 = cmd_hdr.a0;
float a1 = cmd_hdr.a1;
float a2 = cmd_hdr.a2;
int nread = 0;

proc_loop:  for (int i = 0; i < (int)cmd_hdr.n; ++i) {
    // Read next input sample
    auto w = cmd_stream.read();
    float xin = uint_to_float(w.data);

    // Compute the polynomial
    float y = a0 + a1 * xin + a2 * xin * xin;

    // Increment sample count and write output word
    nread++;
    stream_t y_word = axi_word<stream_t>(float_to_uint(y));
    resp_stream.write(y_word);
}
```

Read word

Compute polynomial

Write to output stream

Response Footer and Error Handling

```
// Write the response footer
resp_footer.nread = nread;
resp_footer.err_code = RespFooter::NO_ERR;
resp_footer.stream_write<stream_t>(resp_stream, true);
```

- Check for synchronization error
  - TLAST not in expected position
- If there are errors:
  - Sends response with error code
  - Flush queue to next TLAST

Key design error handling principles:

- *Hardware cannot terminate.*
- *It must self-recover*
- *Must provide visibility to errors*

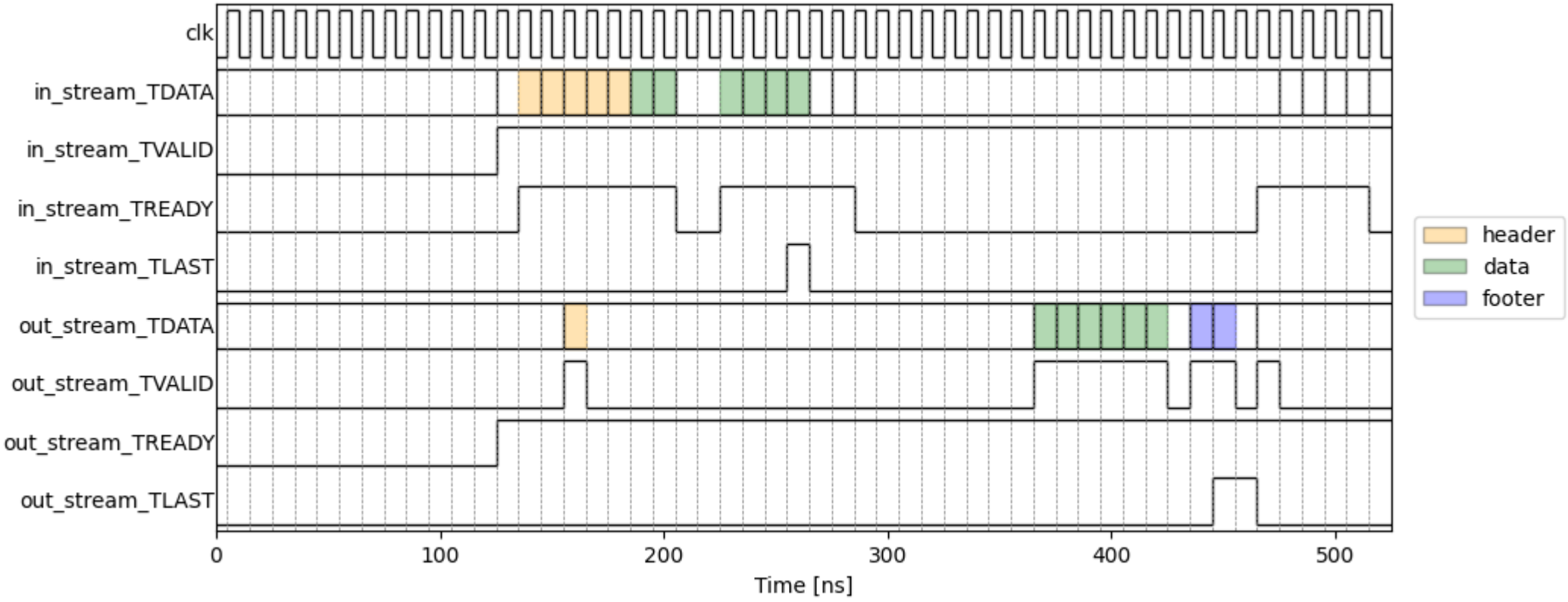
In-Class Demo

We will perform

- ❑C simulation
- ❑C synthesis
- ❑RTL co-simulation
- ❑VCD extraction

```
# ==============================================================
# Vitis HLS TCL script for the block polynomial demo
# ==============================================================

# Create or reset the project
open_project hls_component -reset

# Set the top-level function
set_top poly_block

# --------------------------------------------------------------
# Add design and testbench files
# --------------------------------------------------------------
# Adjust paths if your directory structure differs
add_files poly_block.cpp
add_files -tb tb_poly_block.cpp

# --------------------------------------------------------------
# Create solution and configure hardware part + clock
# --------------------------------------------------------------
open_solution "solution1" -reset
```

Simulation Result



Using an Interface Description Language

- Most professional designers do not hand write their serialization
  - Error prone, tedious
- The class xilinxutils package has a simple interface description language:
  - Serves as a master description for python and Vitis

```
# Command structure
cmd_hdr_fields = [
    FieldInfo("trans_id", IntType(16), descr="Transaction ID"),
    FieldInfo("a0", FloatType(), descr="Coefficient a0"),
    FieldInfo("a1", FloatType(), descr="Coefficient a1"),
    FieldInfo("a2", FloatType(), descr="Coefficient a2"),
    FieldInfo("n", IntType(32, signed=False), descr="Number of data points")]
```

Vitis HLS Code Generation

- The python descriptors can be used to autogenerate the Vitis HLS header files
- No need to manually write the serialization

```python
# Bus widths to support
stream_bus_widths = [32, 64]

# Generate command header include file
cmd_struct = VitisStruct("CmdHdr", datastructs.cmd_hdr_fields)
vg = VitisCodeGen(cmd_struct)
cmd_file = os.path.join(include_dir, "cmd_hdr.h")
vg.gen_include(include_file=cmd_file, bus_widths=stream_bus_widths,
                copy_stream_utils=True)
```

Auto-generated

```cpp
class CmdHdr {
public:

    ap_int<16> trans_id; // Transaction ID
    float a0; // Coefficient a0
    float a1; // Coefficient a1
    float a2; // Coefficient a2
    ap_uint<32> n; // Number of data points

    template<typename Tstream>
    bool stream_read_32(hls::stream<Tstream>& in) {
        constexpr int bus_bits = decltype(Tstream::data)::width;
        static_assert(bus_bits == 32, "Only 32-bit stream supporte

        Tstream w0 = in.read();
        trans_id = w0.data.range(15, 0);
        Tstream w1 = in.read();
        a0 = uint_to_float(w1.data);
        Tstream w2 = in.read();
        a1 = uint_to_float(w2.data);
        Tstream w3 = in.read();
        a2 = uint_to_float(w3.data);
        Tstream w4 = in.read();
        n = w4.data;
        bool tlast = w4.last;

        return tlast;
    }
```